12 Deploying your microservices

This chapter covers

- Understanding why DevOps is critical to microservices
- Configuring the core Amazon infrastructure for O-stock services
- Manually deploying O-stock services to Amazon
- Designing a build/deployment pipeline for your services
- Treating your infrastructure as code
- Deploying your application to the cloud

We're almost at the end of the book, but not the end of our microservices journey. While most of this book has focused on designing, building, and putting Spring-based microservices into operation using the Spring Cloud technology, we haven't touched on how to build and deploy microservices. Creating a build/deployment pipeline might seem like a mundane task, but in reality, it's one of the most critical pieces of our microservices architecture.

Why? Remember, one of the key advantages of a microservices architecture is that microservices are small units of code that can be quickly built, modified, and deployed to a production environment independently of one another. The small size of the service means that new features (and critical bug fixes) can be delivered with a high degree of *velocity*. Velocity is the keyword here because velocity implies that little to no friction exists between creating a new feature or fixing a bug and getting our service redeployed. Lead times for deployment should be minutes, not days. To accomplish this, the mechanism that you use to build and deploy your code needs to be

- *Automated*—When we build our code, there should be no human intervention in the build/deployment process. The process of building the software, provisioning a machine image, and deploying the service should be automated and initiated by the act of committing code to the source repository.
- *Repeatable*—The process that we use to build and deploy our software should be repeatable; the same thing happens every time a build and deploy kicks off. Variability in our process is often the source of subtle bugs that are difficult to track down and resolve.
- *Complete*—The outcome of our deployed artifact should be an entire virtual machine or container image (Docker, for example) that contains the "complete" run-time environment for the service. This is an important shift in the way we think about our infrastructure.
  The provisioning of our machine images needs to be completely automated via scripts and kept under source control along with the service source code. In a microservice environment, this responsibility usually shifts from an operations team to the development team owning the service. One of the core tenets of microservice development is pushing the developers to complete operational responsibility for the service.
- *Immutable*—Once the machine image containing our service is built, the image's run-time configuration should not be touched or changed after deploying the image. If changes need to be made, they need to happen in the scripts kept under source control, and the service and infrastructure must go through the build process again.
  Run-time configuration changes (garbage collection settings, Spring profile used, and so forth) should be passed as environment variables to the image, while application configuration should be kept separate from the container (Spring Cloud Config).

Building a robust and generalized build deployment pipeline is a significant amount of work and is often explicitly designed toward the run-time environment. It usually involves a specialized team of DevOps (developer/operations) engineers, whose sole job is to generalize the build process so that each team can build their microservices without having to reinvent the entire build process. Unfortunately, Spring is a development framework and doesn't offer a significant number of capabilities to implement a build/deployment pipeline. But that doesn't mean that such a pipeline can't be built.

12.1 The architecture of a build/deployment pipeline

The goal of this chapter is to provide you with the working pieces of a build/deployment pipeline so that you can take these pieces and tailor them to your own environment. Let's start our discussion by looking at the general architecture of our build/ deployment pipeline and several of the general patterns and themes that it represents. To keep the examples flowing, we've done a few things that we wouldn't normally do in our own environment, and we'll call those pieces out accordingly.

Our discussion on deploying microservices begins with a picture you saw way back in chapter 1. Figure 12.1 is a duplicate of the diagram you saw in chapter 1, showing the pieces and steps involved in creating a microservices build/deployment pipeline.

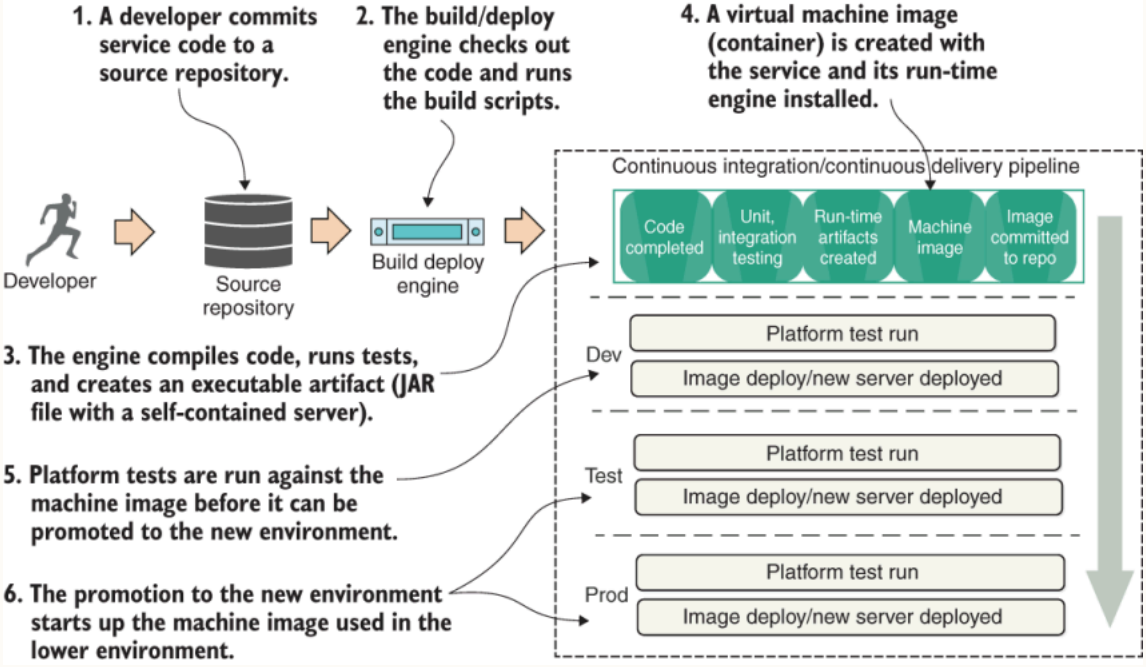


Figure 12.1 Each component in the build/deployment pipeline automates a task that would have been done manually.

Figure 12.1 should look somewhat familiar because it's based on the general build/ deployment pattern used for implementing continuous integration (CI):

1. A developer commits their code to the source code repository.
2. A build tool monitors the source control repository for changes and kicks off a build when a change is detected.
3. During the build, the application's unit and integration tests are run, and if everything passes, a deployable software artifact is created (a JAR, WAR, or EAR).
4. This JAR, WAR, or EAR might then be deployed to an application server running on another server (usually a development server).

A similar process is followed by the build/deployment pipeline until the code is ready to be deployed. We're going to tack continuous delivery (CD) onto the build/deployment process shown in figure 12.1 with the following steps:

1. A developer commits their service code to a source repository.
2. A build/deploy engine monitors the source code repository for changes. If code is committed, the build/deploy engine checks out the code and runs the build scripts.
3. After the build scripts compile the code and run its unit and integration tests, the service is then compiled to an executable artifact. Because our microservices were built using Spring Boot, our build process creates an executable JAR file that contains both the service code and a self-contained Tomcat server.
   Note that the next step is where our build/deploy pipeline begins to deviate from a traditional Java CI build process.
4. After the executable JAR is built, we "bake" a machine image with our microservice deployed to it. This baking process creates a virtual machine image or container (Docker) and installs our service onto it.
   When the virtual machine image starts, our service also starts and is ready to begin taking requests. Unlike a traditional CI build process, where you might deploy the compiled JAR or WAR to an application server that's independently (and often with a separate team) managed from the application, with the CI/CD process, we deploy the microservice, the run-time engine for the service, and the machine image all as one codependent unit managed by the development team that wrote the software.
5. Before we officially deploy to a new environment, the machine image starts, and a series of platform tests are run against the image for the environment to determine if everything is proceeding correctly. If the platform tests pass, the machine image is promoted to the new environment and made available for use.
6. The promotion of the service to the new environment involves starting up the exact machine image that was used in the lower environment. This is the secret sauce of the whole process. The entire machine image is deployed.

With CD, no changes are made to any installed software (including the operating system) after the server is created. By promoting and using the same machine image, you guarantee the immutability of the server as it moves from one environment to the next.

Unit tests vs. integration tests vs. platform tests

Figure 12.1 exposes several types of testing (unit, integration, and platform) during the building and deployment of a service. Three types of testing are typical in this type of a pipeline:

- *Unit tests*—These are run immediately before the compilation of the service code, but before deployment to an environment. The tests are designed to run in complete isolation, with each unit test being small and narrow in focus. A unit test should have no dependencies on third-party infrastructure databases, services, and so on. Usually, a unit test scope encompasses the testing of a single method or function.
- *Integration tests*—These are run immediately after packaging the service code. They are designed to test an entire workflow, code path, and stub or to mock primary services or components that would need to be called with third-party services. During integration testing, you might be running an in-memory database to hold data, mocking out third-party service calls, and so on. With integration tests, third-party dependencies are mocked or stubbed so that any requests that would invoke a remote service are also mocked or stubbed. The calls never leave the build server.
- *Platform tests*—These are run right before a service is deployed to an environment. These tests typically test an entire business flow and also call all the third-party dependencies that would normally be called in a production system. Platform tests are running live in a particular environment and don't involve any mocked-out services. This type of test is run to determine integration problems with third-party services that would typically not be detected when a third-party service is stubbed out during an integration test.

If you are interested in diving into more detail on how to create unit, integration, and platform tests, we highly recommend Testing Java Microservices (Manning, 2018) by Alex Soto Bueno, Andy Gumbrecht, and Jason Porter.

The build/deploy process is built on four core patterns. These patterns have emerged from the collective experience of development teams building microservice and cloud-based applications. These patterns include

- *Continuous integration/continuous delivery (CI/CD)*—With CI/CD, your application code isn't only built and tested when it is committed and deployed. The deployment of your code should go something like this: if the code passes its unit, integration, and platform tests, it's immediately promoted to the next environment. The only stopping point in most organizations is the push to production.
- *Infrastructure as code*—The final software artifact that's pushed to development and beyond is a machine image. The machine image with your microservice installed in it is provisioned immediately after your microservice's source code is compiled and tested. The provisioning of the machine image occurs through a series of scripts that run with each build. No human hands should ever touch the server after it's built. The provisioning scripts should be kept under source control and managed like any other piece of code.
- *Immutable server*—Once a server image is built, the server's configuration and microservice are never touched after the provisioning process. This guarantees that your environment won't suffer from "configuration drift," where a developer or system administrator made "one small change" that later caused an outage. If a change needs to be made, change the provisioning scripts for the server and kick off a new build.

On immutability and the rise of the Phoenix Server

With the concept of immutable servers, we should always be guaranteed that a server's configuration matches precisely with what the machine image for the server says it does. A server should have the option to be killed and restarted from the machine image without any changes in the service or microservice behavior. This killing and resurrection of a new server was termed "Phoenix Server" by Martin Fowler because, when the old server is killed, the new server should rise from the ashes. For more information, see the following link:

- **http://martinfowler.com/bliki/PhoenixServer.html**

The Phoenix Server pattern has two fundamental benefits. First, it exposes and drives configuration drift from your environment. If you're constantly tearing down and setting up new servers, you're more likely to expose configuration drift early. This is a tremendous help in ensuring consistency.

Second, the Phoenix Server pattern helps to improve resiliency by allowing you to find situations where a server or service isn't cleanly recoverable after it's killed and restarted. Remember, in a microservices architecture, your services should be stateless, and the death of a server should be a minor blip. Randomly killing and restarting servers quickly exposes situations where you have a state in your services or infrastructure. It's better to find these situations and dependencies early in your deployment pipeline than when you're on the phone with an angry customer or company.

For this chapter, we'll see how to implement a build/deployment pipeline using several non-Spring tools. We're going to take the suite of microservices we've built for this book and do the following:

1. Integrate our Maven build scripts into a CI/deployment cloud-based tool called Jenkins
2. Build immutable Docker images for each service and push those images to a centralized repository
3. Deploy the entire suite of microservices to Amazon's cloud using Amazon's container service called the Elastic Kubernetes Service (EKS)

NOTE In this book, we won't explain in detail how Kubernetes works. In case you are new or want to learn more about how it works, we highly recommend you read Marko Lukša's excellent book, *Kubernetes in Action* (Manning, 2017), which covers the subject in detail.

Before we start with our pipeline, let's begin by setting up the core infrastructure in the cloud.

12.2 Setting up O-stock's core infrastructure in the cloud

Throughout all the code examples in this book, we've run our applications inside a single virtual machine image with each individual service running as a Docker container. We're going to change that now by separating our database server (PostgreSQL) and caching server (Redis) away from Docker into the Amazon cloud. All the other services will remain running as Docker containers inside a single-node Amazon EKS cluster. Figure 12.2 shows the deployment of the O-stock services to the Amazon cloud.

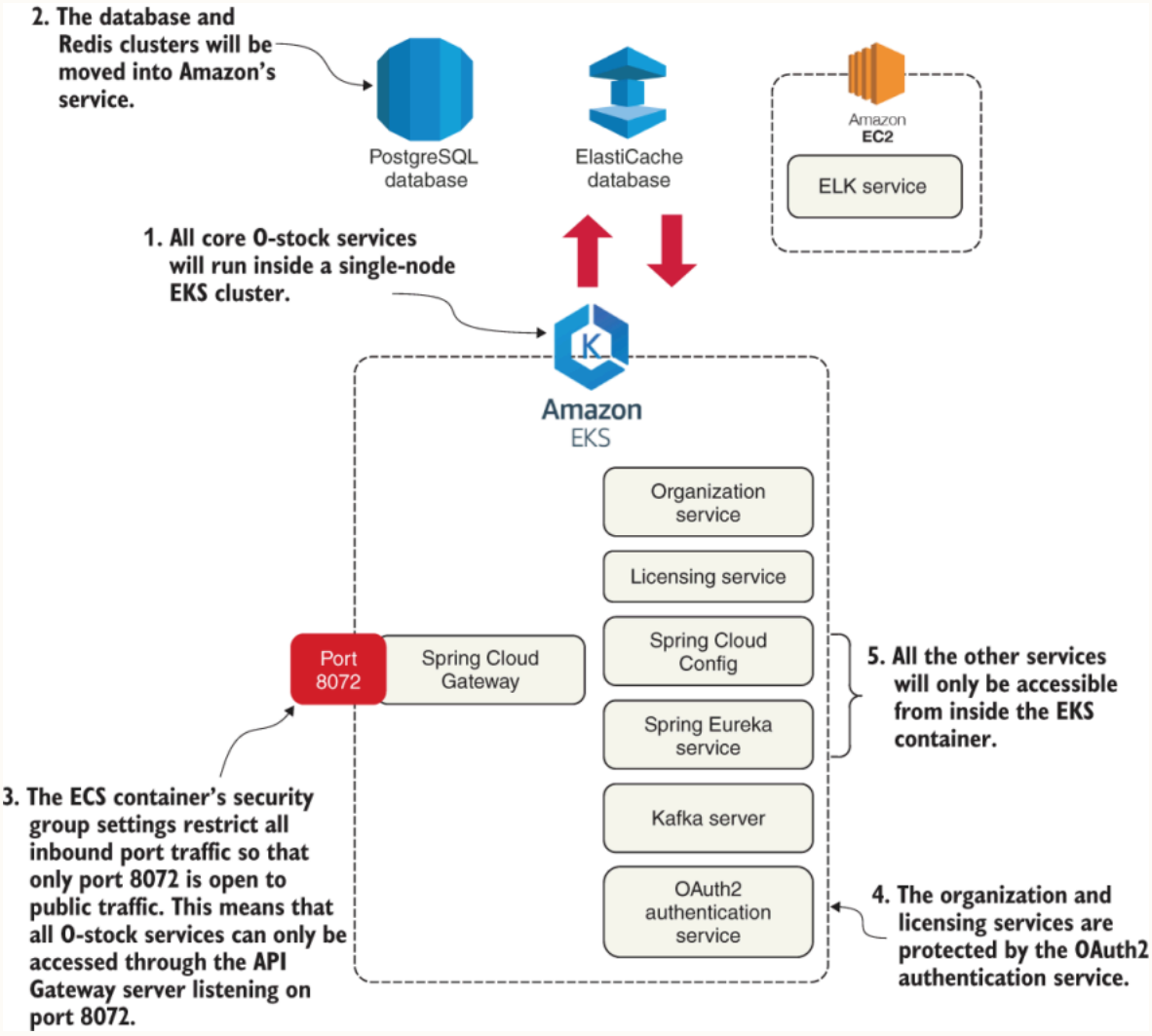


Figure 12.2 With Docker, we can deploy all our services to a cloud provider such as Amazon EKS.

Let's walk through the details in figure 12.2. Note that the numbers correspond to those in the figure:

1. All our O-stock services (minus the database and the Redis cluster) will be deployed as Docker containers running inside a single-node EKS cluster. EKS configures and sets up the servers needed to run a Docker cluster. EKS can also monitor the health of containers running in Docker and restart our service if it crashes.
2. With the deployment to the Amazon cloud, we'll move away from using our PostgreSQL database and Redis server and instead use the Amazon Relational Database Service (Amazon RDS) and Amazon ElastiCache services. We could continue to run the Postgres and Redis data stores in Docker, but we wanted to highlight how easy it is to move from one infrastructure to another that's managed completely by a cloud provider.
3. Unlike our desktop deployment, we want all traffic for the server to go through our API gateway. We're going to use an Amazon security group to only allow port 8072 on the deployed EKS cluster to be accessible to the world.
4. We'll still use Spring's OAuth2 server to protect our services. Before the organization and licensing services can be accessed, the user needs to authenticate with our authentication services (see chapter 9 for details on this) and present a valid OAuth2 token on every service call.
5. All our servers, including our Kafka server, won't be publicly accessible to the outside world via their exposed Docker ports. They will only be available from inside an EKS container.

Some prerequisites for working with AWS

To set up your Amazon infrastructure, you'll need the following:

- *Your own Amazon Web Services (AWS) account*—You should have a basic understanding of the AWS console and the concepts behind working in this environment.
- A web browser
- *The AWS CLI (command-line interface)*—This unified tool manages AWS Services. See **https://docs.aws.amazon.com/cli/latest/userguide/install-cliv2.html**.
- *Kubectl*—This tool allow us to communicate and interact with our Kubernetes cluster. See **https://kubernetes.io/docs/tasks/tools/install-kubectl/#install-kubectl**.
- *IAM Authenticator*—This tool provides authentication to our Kubernetes cluster. See **https://docs.aws.amazon.com/eks/latest/userguide/install-aws-iam-authenticator.html**.
- *Eksctl*—A simple command-line utility for managing and creating AWS EKS clusters in our AWS account. See **https://docs.aws.amazon.com/eks/latest/userguide/getting-started-eksctl.html**.

If you're entirely new to AWS, we highly recommend you pick up a copy of Michael and Andreas Wittig's book, Amazon Web Services in Action (Manning, 2018). The first chapter in this book includes a well-written tutorial at the end of the chapter on how to sign up and configure your AWS account. That chapter is available for download here:

https://www.manning.com/books/amazon-web-services-in-action-second-edition

In this chapter, we've tried as much as possible to use the free-tier services offered by Amazon. The only place where we couldn't do this was when setting up the EKS cluster. We used an M4 large server that costs approximately $0.10 per hour to run. If you don't want to incur high costs running this server, ensure that you shut down your services after you're done. If you want to know more about AWS EC2 instance pricing, visit the official Amazon documentation at the following link: **https://aws.amazon.com/ec2/pricing/on-demand/**.

And finally, there's no guarantee that the Amazon resources (Postgres, Redis, and EKS) that we use in this chapter will be available if you want to run this code yourself. If you're going to run the code from this chapter, you need to set up your own GitHub repository (for your application configuration), your own Jenkins environment, Docker Hub (for your Docker images), and Amazon account. You'll then need to modify your application configuration to point to your account and credentials.

12.2.1 Creating the PostgreSQL database using Amazon RDS

To begin, we need to set up and configure our Amazon AWS account. Once this is done, we'll create the PostgreSQL database that we'll use for our O-stock services. To do this, we first need to log in to the Amazon AWS Management Console (**https://aws.amazon.com/console/**). You'll be presented with a list of Amazon web services when you first log into the console.

1. Locate the link called RDS. Click the link, which takes you to the RDS dashboard.
2. On the RDS dashboard, click the big Create Database button.
3. You should see a list of databases. Although Amazon RDS supports different database engines, select PostgreSQL and click Option. This displays the database fields.

You should now see a screen with three template options: a production database, a dev/test database, and a free tier. Select the Free Tier option. Next, under Settings, add the necessary information about our PostgreSQL database, setting the master user ID and password that we'll use to log in to the database. Figure 12.3 shows this screen.

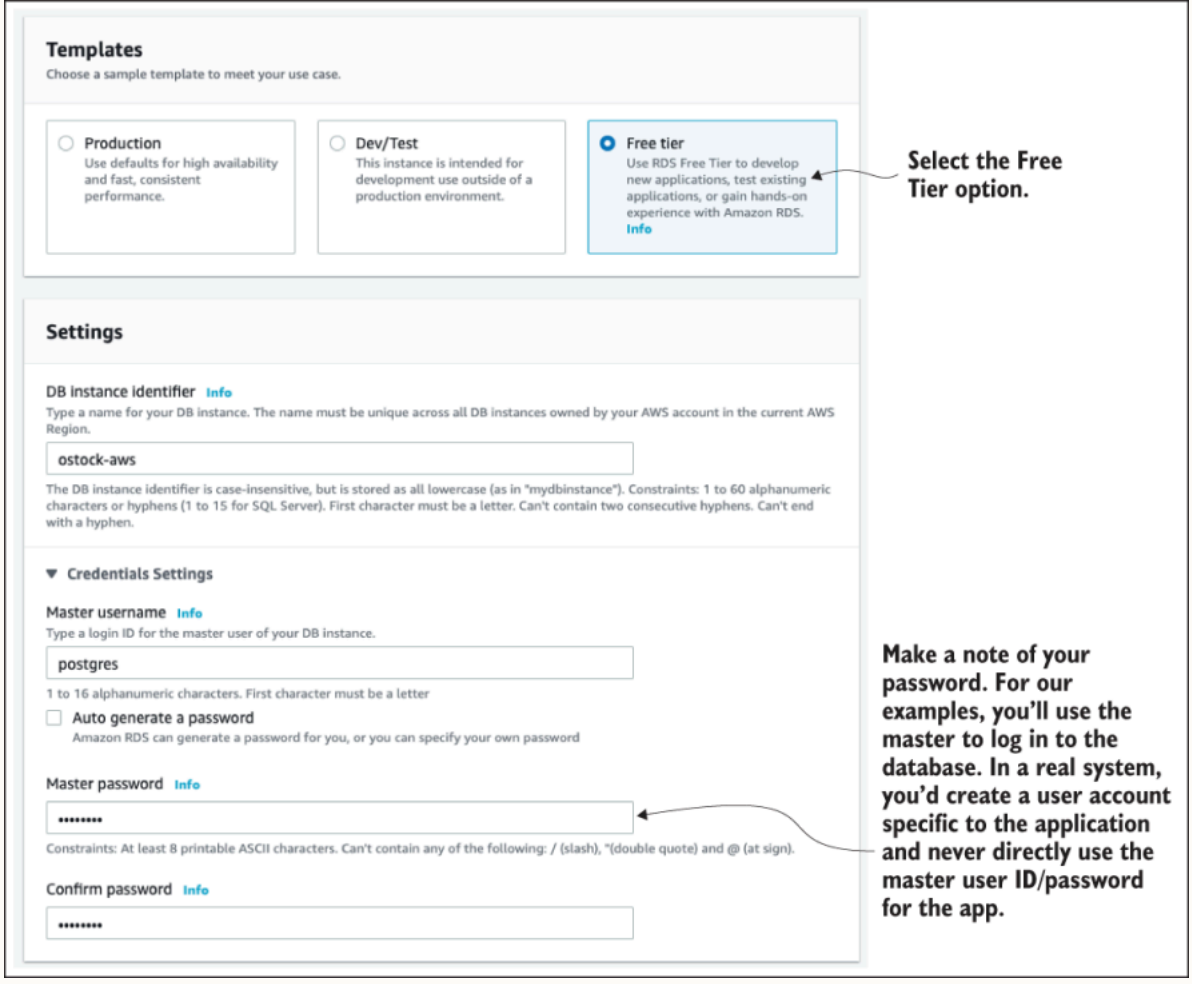


Figure 12.3 Selecting whether our database is going to use a production, test, or free tier template and setting up the basic database configuration in Amazon's AWS Management Console

Next, we'll leave the default configurations for these sections: DB Instance Size, Storage, Availability & Durability, and Database Authentication. The final step is to set the following options. At any time you can click Info to get help about your selection. Figure 12.4 shows this screen.

- *Virtual Private Cloud (VPC)*—Choose Default VPC.
- *Publicly Accessible*—Select Yes.
- *VPC Security Group*—Select Create New, and add a name for the group. For this example, we named the security group ostock-sg.
- *Database Port*—Enter `5432` for the TCP/IP port.

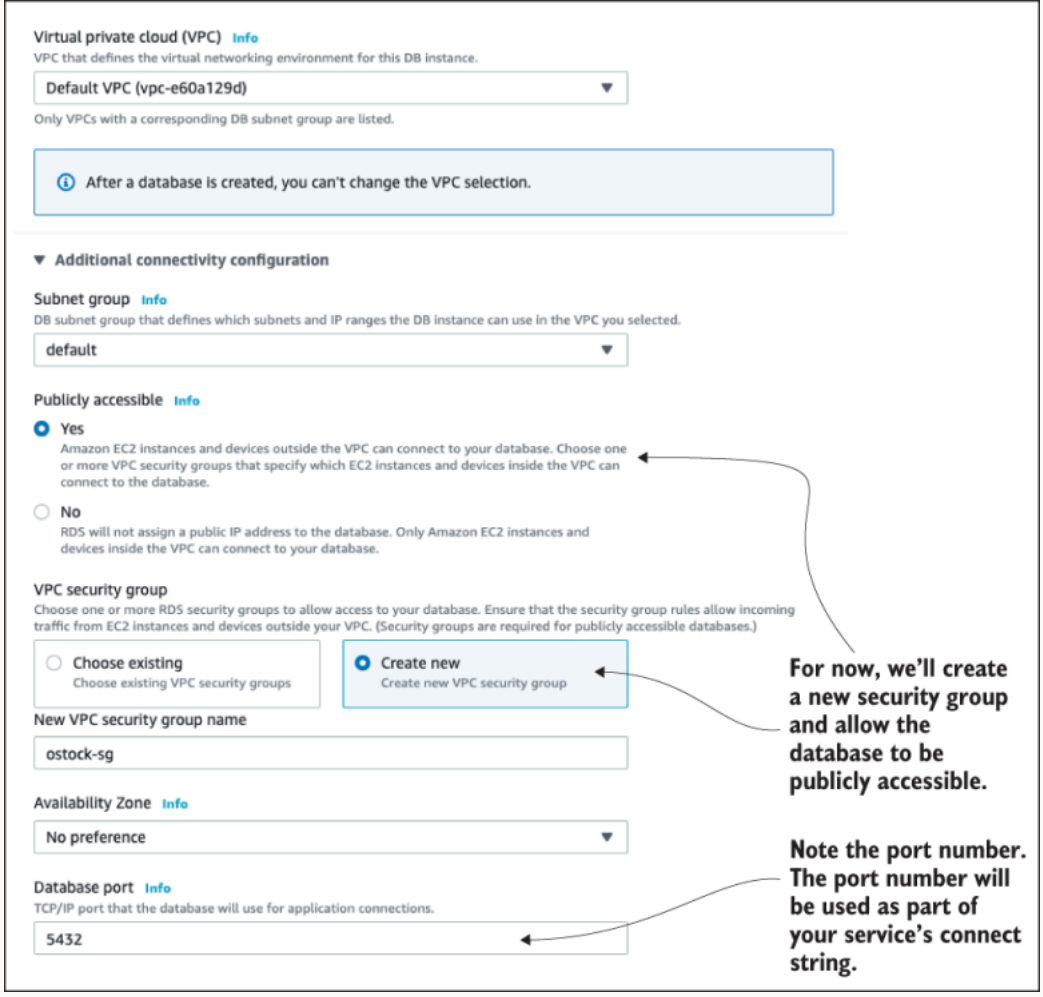


Figure 12.4 Setting up the networking environment, security group, and port number for the RDS database

At this point, our database creation process begins (it can take several minutes). Once it's done, we'll need to configure the O-stock service properties to use the database in the configuration files for the service located at the Spring Configuration Server. To do that, we'll navigate back to the RDS dashboard to view information about our database. Figure 12.5 shows this screen.

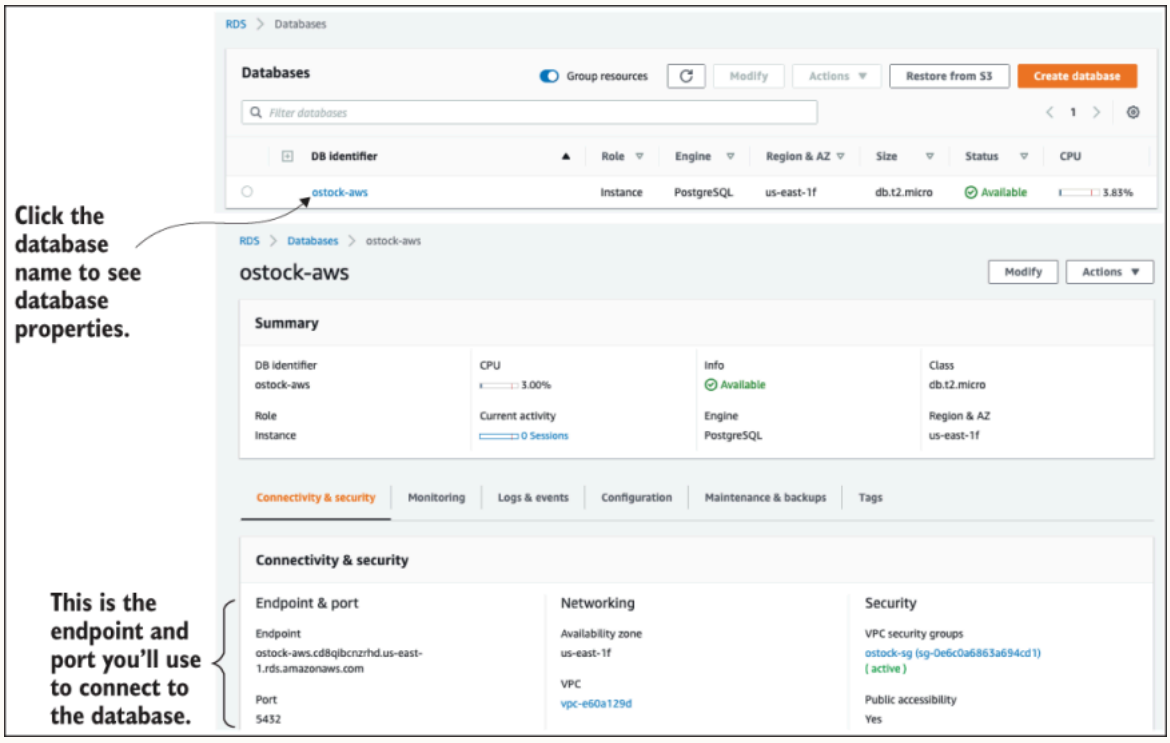


Figure 12.5 Viewing the newly created Amazon RDS/PostgreSQL database and its properties

For this chapter, you can create a new application profile called aws-dev for each microservice that needs to access the Amazon-based PostgreSQL database. The application profile will contain Amazon database connection information.

At this point, our database is ready to go (not bad for setting it up in approximately five clicks). Let's move to the next piece of application infrastructure and see how to create the Redis cluster that our O-stock licensing service is going to use.

12.2.2 Creating the Redis cluster in Amazon

For the O-stock services, we want to move the Redis server we were running in Docker to Amazon ElastiCache. ElastiCache (**https://aws.amazon.com/elasticache/**) lets us build in-memory data caches using Redis or memcached (**https://memcached.org/**) data stores.

To begin, let's navigate back to the AWS Management Console's main page, then search for the ElastiCache service and click the ElastiCache link. From the ElastiCache console, select the Redis link (left side of the screen), and then click the blue Create button at the top of the screen. This opens the ElastiCache/Redis creation wizard (figure 12.6).

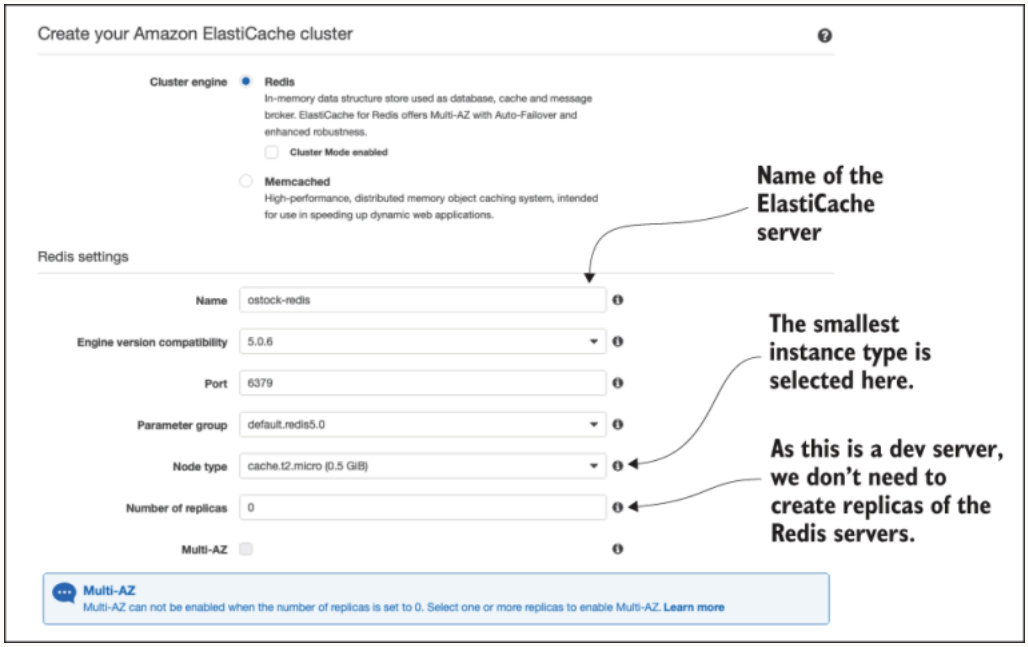


Figure 12.6 With a few clicks, we can set up a Redis cluster whose infrastructure is managed by Amazon.

As for the Advance Redis settings on this page, let's select the same security group we created for the PostgreSQL database, then uncheck the Enable Automatic Backups option. Once the form is filled to your liking, go ahead and click the Create button. Amazon begins the Redis cluster creation process (this will take several minutes). Amazon builds a single-node Redis server running on the smallest Amazon server instance available.

Once our Redis cluster is created, we can click the name of the cluster, and it will take us to a detailed screen showing the endpoint used in the cluster (figure 12.7).

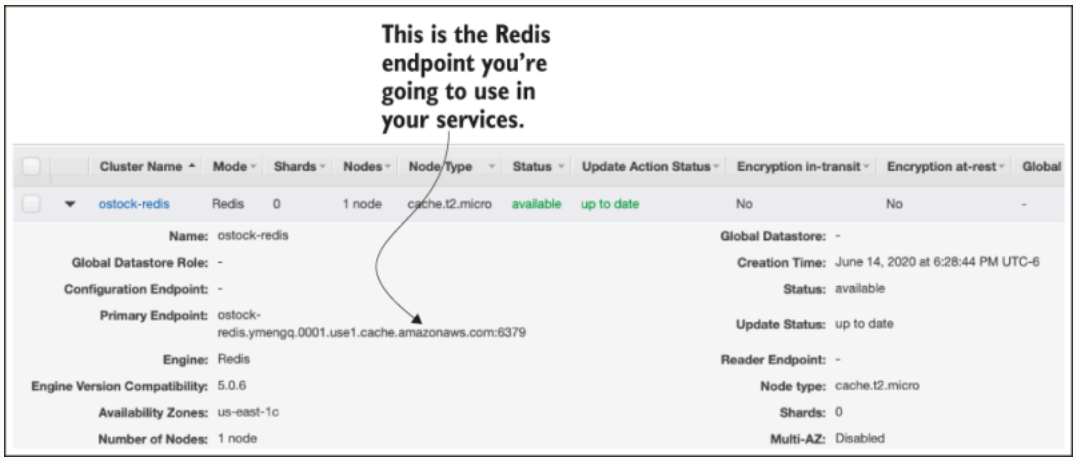


Figure 12.7 The primary endpoint is the key piece of information your services need to connect to Redis.

The licensing service is the only one of our services to use Redis. If you deploy the code examples in this chapter to your own Amazon instance, you must modify the licensing service's Spring Cloud Config files appropriately.

12.3 Beyond the infrastructure: Deploying O-stock and ELK

In this second part of the chapter, you're going to deploy the Elasticsearch, Logstash, and Kibana (ELK) services to an EC2 instance, and the O-stock services to an Amazon EKS container. Remember, an EC2 instance is a virtual server in Amazon's Elastic Compute Cloud (Amazon EC2) where we can run applications. We'll divide the O-stock deployment process into two parts as well.

The first part of your work is for the terminally impatient (like us), showing how to deploy O-stock manually to your Amazon EKS cluster. This will help you understand the mechanics of deploying the service and see the deployed services running in your container. While getting your hands dirty and manually deploying your services is fun, it isn't sustainable or recommended.

In the second part, you'll automate the entire build/deployment process and take the human out of the picture. This is your targeted end state and really tops the work you've done in this book by demonstrating how to design, build, and deploy microservices to the cloud.

12.3.1 Creating an EC2 with ELK

To set up the ELK services, we'll use an Amazon EC2 instance. Having this service separated from the ones we'll deploy to the ELK instance shows you that we can have different instances and still use the services. To create the EC2 instances, we'll carry out the following steps:

1. Choose an Amazon Machine Image (AMI)
2. Select an instance type
3. Set the default configuration for the configuration details, storage, and tags
4. Create a new security group
5. Launch the EC2 instance

To begin, navigate back to the AWS Management Console's main page, search for the EC2 service, and click the link. Next, click the blue Launch Instance button. This brings up the EC2 launch instance. This wizard contains seven different steps, and we will guide you through each of them. Figure 12.8 shows the first step in the EC2 creation process.



Figure 12.8 Select the Amazon Linux 2 AMI option to set up an ELK service instance.

Next, we'll choose the instance type. For this example, select an m4.large server to give us 8 GB of memory and a low hourly cost ($0.10 per hour). Figure 12.9 shows our second step.



Figure 12.9 Selecting the instance type for the ELK service

Once you've selected the instance, click the Next: Configure Instance link. In the third, fourth, and fifth steps, you don't need to change anything, so just click Next through those steps, accepting all the default configurations.

For the sixth step, we need to select to create a new security group (or select an existing Amazon security group if you've already created one) to apply to the new EC2 instance. For this example, we will allow all inbound traffic (0.0.0.0/0 is the network mask for the

entire World Wide Web). In real world scenarios, you need to be careful with this setting, so we recommend that you take some time to analyze the inbound rules that best fit your needs. Figure 12.10 shows this step.



Figure 12.10 Creating a security group for the EC2 instance

Once the security group is set, click the Review and Launch button to check the EC2 instance configuration. Once you're satisfied that the configuration is correct, click Launch Instances to execute the last step. The final step involves the creation of a key pair to connect to the EC2 instance. Figure 12.11 shows this process.

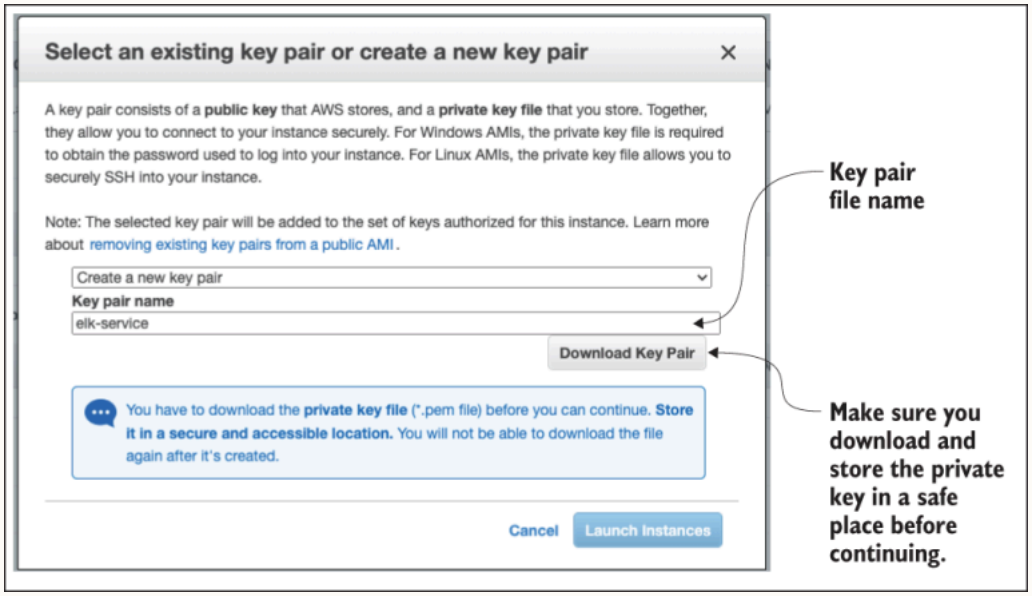


Figure 12.11 Creating a key pair for our EC2 instance security group

Now we have all the infrastructure we need to run the ELK Stack. Figure 12.12 shows the status of our EC2 instance.

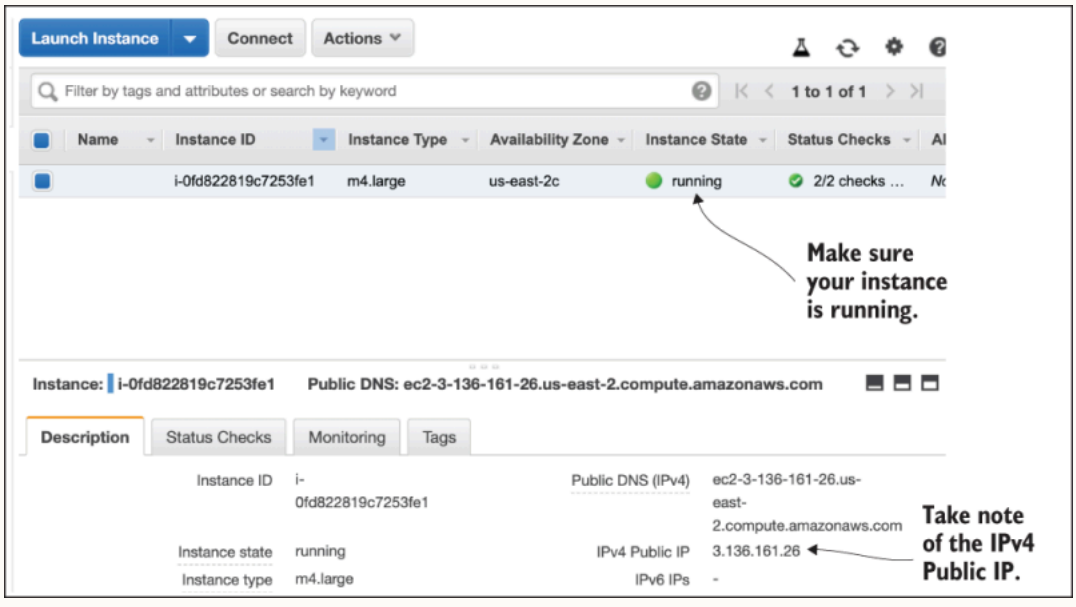


Figure 12.12 Amazon EC2 dashboard page showing the status of our newly created EC2 instance

Before moving on to the next step, take note of the IPV4 instance state. We'll need that information to connect to our EC2 instance.

NOTE Before you continue with the next part, we highly recommend you download the AWS folder from the chapter 12 repository here: https://github.com/ihuaylupo/manning-smia/tree/master/chapter12/AWS. In this repository, we've already created the service scripts in the .yaml and docker-compose files that we'll use for deploying the EC2 and EKS containers.

To continue, open a terminal console on the path where you downloaded the key-pair file and execute the following commands:

```
chmod 400 <pemKey>.pem
scp -r -i <pemKey>.pem ~/project/folder/from/root ec2-user@<IPv4>:~/
```

It's important to highlight that we are going to use a project folder that contains all of the configuration files we need to deploy our services. If you downloaded the AWS folder from GitHub, you can use that as your project folder. Once the previous commands are executed, you should see the output shown in figure 12.13.

```
Last login: Sat Jun 20 06:35:47 2020 from 186.176.131.131

       __|  __|_  )
       _|  (     /   Amazon Linux AMI
      ___|\___|___|

https://aws.amazon.com/amazon-linux-ami/2018.03-release-notes/
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
[ec2-user@ip-172-31-42-196 ~]$ ls -l
total 4
drwxr-xr-x 3 ec2-user ec2-user 4096 Jun 18 16:55 AWS
```

Figure 12.13 Now we can log into the EC2 instance using the ssh and key pair previously created.

If we list the files and directories of the root for our EC2 instance, we will see that the entire project folder was pushed into the instance. Now that everything is set up, let's continue with the ELK deployment.

12.3.2 Deploying the ELK Stack in the EC2 instance

Now that we have created the instance and logged in, let's continue with the deployment of the ELK services. To achieve this, we must complete the following steps:

1. Update the EC2 instance
2. Install Docker
3. Install Docker Compose
4. Run Docker
5. Run the docker-compose file that contains the ELK services

To execute all of the previous steps, run the commands shown in the following listing.

Listing 12.1 ELK deployment in an EC2 instance

```
sudo yum update                                          (1)
sudo yum install docker                                  (2)
sudo curl -L https://github.com/docker/compose/releases/
    download/1.21.0/docker-compose-`uname
    -s`-`uname -m` | sudo tee /usr/local/bin/
    docker-compose > /dev/null                           (3)
sudo chmod +x /usr/local/bin/docker-compose
sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose
sudo service docker start                                (4)
sudo docker-compose -f AWS/EC2/
    docker-compose.yml up -d                             (5)
```

(1) Updates the applications installed on a system (in our scenario, the EC2 instance)

(2) Installs Docker

(3) Installs Docker Compose

❹ Runs Docker in the EC2 instance

❺ Runs the docker-compose file. The -d option runs the process in the background.

Now that we have all the ELK services running, to verify that everything is up, you can enter the http://<IPv4>:5601/ URL. You'll see the Kibana dashboard. Now that we have the ELK services running, let's create an Amazon EKS cluster to deploy our microservices.

12.3.3 Creating an EKS cluster

The last and final step before deploying the O-stock services is to set up an Amazon EKS cluster. EKS is an Amazon service that allows us to run Kubernetes on AWS. Kubernetes is an open source system that automates deployment, scheduling, and the creation and deletion of containers. As previously discussed, managing a large number of containers and microservices can become a challenging task. With tools like Kubernetes, we can handle all of our containers in a faster, more efficient way. For example, let's imagine the following scenario.

We need to update a microservice and its image, which is propagated in dozens or maybe hundreds of containers. Can you imagine yourself manually deleting and re-creating those containers? With Kubernetes, you can destroy and re-create them simply by executing a short command. This is only one of the numerous advantages that Kubernetes brings to the table. Now that we have a brief overview of what Kubernetes is and does, let's continue.

The process of creating the EKS cluster can be done by using the AWS Management Console or the AWS CLI. For this example, we will show you how to use the AWS CLI.

NOTE To continue with this part, make sure you have all the tools previously mentioned in the earlier sidebar entitled, "Some prerequisites for working with AWS," and that you have configured the AWS CLI. In case you haven't, we highly recommend you look at the documentation here: **https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-quickstart.html**.

After we've configured and installed all the necessary tools, we can start creating and configuring our Kubernetes cluster. To do so, we need to

1. Provision a Kubernetes cluster
2. Push the microservice images into repositories
3. Deploy our microservices manually to the Kubernetes cluster

PROVISIONING A KUBERNETES CLUSTER

To create our cluster using the AWS CLI, we need to use eksctl. This is accomplished by using the following command:

```
eksctl create cluster --name=ostock-dev-cluster
  --nodes=1 --node-type=m4.large
```

The previous eksctl command line allows us to create a Kubernetes cluster (named ostock-dev-cluster), which uses a single m4.large EC2 instance as a worker node. In this scenario, we use the same instance type we previously selected in the EC2 ELK service creation wizard. The execution of this command takes several minutes. While processing, you'll see different line outputs, but you will know when it is finished because you'll see the following output:

```
[✓] EKS cluster "ostock-dev-cluster" in "region-code" region is ready
```

Once executed, you can now run the `kubectl get svc` command to verify that the Kubectl configuration is correct. This command should show the following output:

```
NAME         TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.100.0.1   <none>      443/TCP   1m
```

PUSHING THE MICROSERVICE IMAGES INTO REPOSITORIES

Up to now, we've built our services on our local machine. To use these images in our EKS cluster, we need to push the Docker container images to a container repository. A *container repository* is like a Maven repository for your created Docker images. Docker images can be tagged and uploaded to it, then other projects can download and use the images.

There are several repositories such as the Docker Hub, but for this example, let's continue with the AWS infrastructure and use the Amazon Elastic Container Registry (ECR). To learn more about ECR, see https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html.

To push the images to the container registry, we must ensure the Docker images are in our local Docker directory. If they aren't there, execute the following command on the parent pom.xml file:

```
mvn clean package dockerfile:build
```

Once the command executes, we should now see the images in the Docker images list. To verify this, issue the `docker images` command. You should see the following output:

```
REPOSITORY                  TAG        IMAGE ID       SIZE
ostock/organization-service    chapter12   b1c7b262926e    485MB
ostock/gatewayserver           chapter12   61c6fc020dcf    450MB
ostock/configserver            chapter12   877c9d855d91    432MB
ostock/licensing-service       chapter12   6a76bee3e40c    490MB
ostock/authentication-service   chapter12   5e5e74f29c2     452MB
ostock/eurekaserver            chapter12   e6bc59ae1d87    451MB
```

The next step is to authenticate our Docker client with our ECR registry. To achieve this, we need to obtain a password and our AWS account ID by executing the following commands:

```
aws ecr get-login-password
aws sts get-caller-identity --output text --query "Account"
```

The first command retrieves the password, and the second returns our AWS account. We'll use both values to authenticate our Docker client. Now that we have our credentials, let's authenticate by executing the following command:

```
docker login -u AWS -p [password]
↪ https://[aws_account_id].dkr.ecr.[region].amazonaws.com
```

Once authenticated, the next step is to create the repositories where we will store our images. To create these repositories, let's execute the following commands:

```
aws ecr create-repository --repository-name ostock/configserver
aws ecr create-repository --repository-name ostock/gatewayserver
aws ecr create-repository --repository-name ostock/eurekaserver
aws ecr create-repository --repository-name ostock/authentication-service
aws ecr create-repository --repository-name ostock/licensing-service
aws ecr create-repository --repository-name ostock/organization-service
```

When each command executes you should see output similar to the following:

```
{
   "repository": {
      "repositoryArn": "arn:aws:ecr:us-east-2:
    8193XXXXXXX43:repository/ostock/configserver",
      "registryId": "8193XXXXXXX43",
      "repositoryName": "ostock/configserver",
      "repositoryUri": "8193XXXXXXX43.dkr.ecr.
    us-east-2.amazonaws.com/ostock/configserver",
      "createdAt": "2020-06-18T11:53:06-06:00",
      "imageTagMutability": "MUTABLE",
      "imageScanningConfiguration": {
         "scanOnPush": false
      }
   }
}
```

Make sure to take note of all the repository URIs because we are going to need them to create the tag and push the images. To create the tags for our images, let's execute the following commands:

```
docker tag ostock/configserver:chapter12 [configserver-repository-uri]:chapter12
docker tag ostock/gatewayserver:chapter12 [gatewayserver-repository-uri]:chapter12
docker tag ostock/eurekaserver:chapter12 [eurekaserver-repository-uri]:chapter12
docker tag ostock/authentication-service:chapter12 [authentication-service-repository-uri]:chapter12
docker tag ostock/licensing-service:chapter12 [licensing-service-repository-uri]:chapter12
docker tag ostock/organization-service:chapter12 [organization-service-repository-uri]:chapter12
```

The `docker tag` command creates a new tag for our images. If you execute this command, you should see output similar to that shown in figure 12.14.

```
REPOSITORY                                                                  TAG         IMAGE ID       CREATED          SIZE
819322222443.dkr.ecr.us-east-2.amazonaws.com/ostock/organization-service    chapter12   b1c7b262926e   45 hours ago     485MB
ostock/organization-service                                                 chapter12   b1c7b262926e   45 hours ago     485MB
819322222443.dkr.ecr.us-east-2.amazonaws.com/ostock/gatewayserver           chapter12   61c6fc020dcf   45 hours ago     450MB
ostock/gatewayserver                                                        chapter12   61c6fc020dcf   45 hours ago     450MB
819322222443.dkr.ecr.us-east-2.amazonaws.com/ostock/configserver            chapter12   877c9d855d91   45 hours ago     432MB
ostock/configserver                                                         chapter12   877c9d855d91   45 hours ago     432MB
819322222443.dkr.ecr.us-east-2.amazonaws.com/ostock/licensing-service       chapter12   6a76bee3e40c   46 hours ago     490MB
ostock/licensing-service                                                    chapter12   6a76bee3e40c   46 hours ago     490MB
819322222443.dkr.ecr.us-east-2.amazonaws.com/ostock/authentication-service  chapter12   f5e5e74f29c2   5 days ago       452MB
ostock/authentication-service                                               chapter12   f5e5e74f29c2   5 days ago       452MB
819322222443.dkr.ecr.us-east-2.amazonaws.com/ostock/eurekaserver            chapter12   e6bc59ae1d87   5 days ago       451MB
ostock/eurekaserver                                                         chapter12   e6bc59ae1d87   5 days ago       451MB
```

Figure 12.14 Docker images with the Amazon Elastic Container Registry (ECR) repository URI tags

Finally, we can push our microservice images to the ECR registry repositories. To do this, we'll execute the following commands:

```
docker push [configserver-repository-uri]:chapter12
docker push [gatewayserver-repository-uri]:chapter12
docker push [eurekaserver-repository-uri]:chapter12
docker push [authentication-service-repository-uri]:chapter12
docker push [licensing-service-repository-uri]:chapter12
docker push [organization-service-repository-uri]:chapter12
```

Once all the commands run, you can visit the ECR service in the AWS Management Console. You should see a list similar to the one shown in figure 12.15.

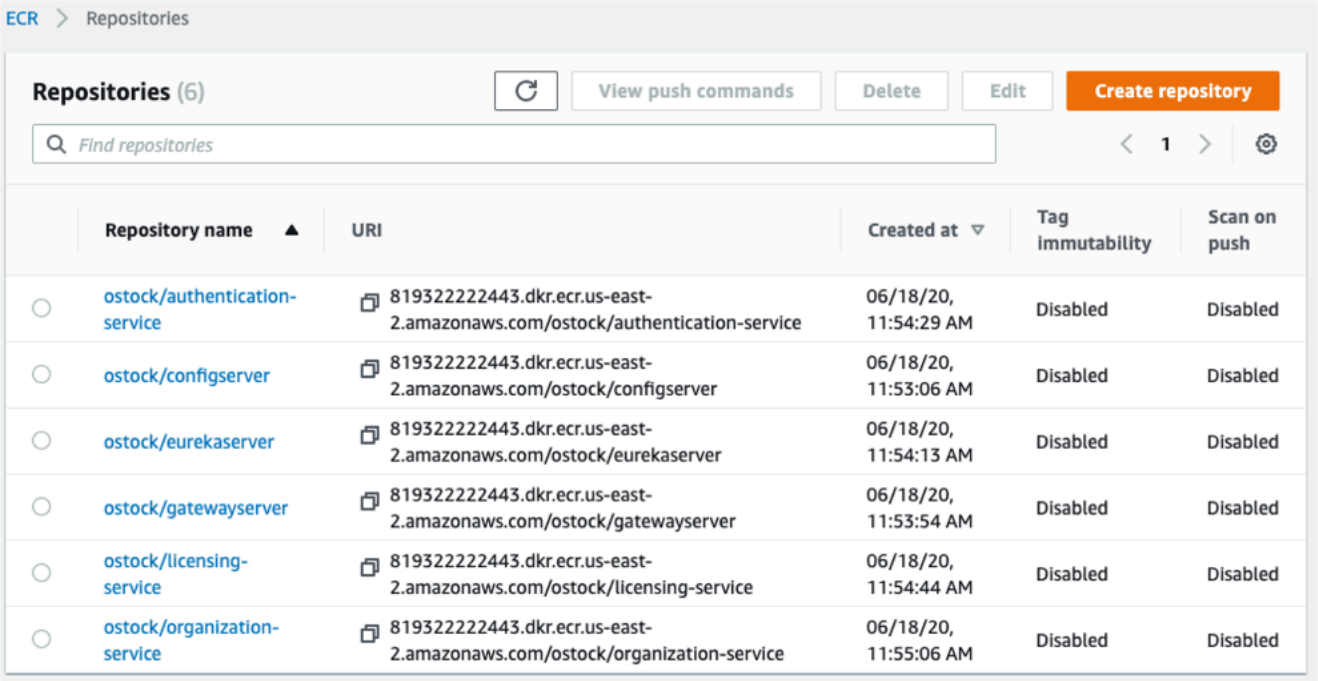


Figure 12.15 ECR repositories for the examples in this chapter

DEPLOYING OUR MICROSERVICES IN THE EKS CLUSTER

To deploy our microservices, we first need to make sure that the licensing, organization, and gateway services have the appropriate Spring Cloud Config files. Remember, the PostgreSQL, Redis, and the ELK Stack services changed.

NOTE In the GitHub repository, you'll find all the changes we made to the configuration server, which also contains the configuration for the Logstash server. Here's the link to GitHub: **https://github.com/ihuaylupo/manning-smia/tree/master/chapter12**. Remember, we used a hardcoded value when we created the Logstash configuration.

Before deploying our microservices, let's create our Kafka and Zookeeper services. There are several ways to create these services. For this scenario we chose to create them using Helm charts. In case you are not familiar with Helm, it's a package manager for Kubernetes clusters. A Helm chart is a Helm package that contains all the required resource definitions to run a specific service, application, or tool inside a Kubernetes cluster.

NOTE If you don't have Helm installed, visit **https://helm.sh/docs/intro/install/** to see all the possible ways to install it on your computer.

Once Helm is installed, let's execute the following commands to create our Kafka and Zookeeper services:

```
helm install zookeeper bitnami/zookeeper \
  --set replicaCount=1 \
  --set auth.enabled=false \
  --set allowAnonymousLogin=true

helm install kafka bitnami/kafka \
  --set zookeeper.enabled=false \
  --set replicaCount=1 \
  --set externalZookeeper.servers=zookeeper
```

Once these commands execute, you should see output with some of the details of the services. To be sure that everything ran successfully, execute the `kubectl get pods` command to see your running services.

```
NAME       READY   STATUS    RESTARTS   AGE
kafka-0    1/1     Running   0          77s
zookeeper-0  1/1     Running   0          101s
```

The next step in deploying our services is to convert our docker-compose file into a compatible Kubernetes format. Why? Kubernetes doesn't support the Docker Compose files, so in order to execute these files, we need to use a tool called Kompose.

Kompose converts Docker Compose files to container implementations such as Kubernetes. Using this tool allows us to convert all the docker-compose.yaml file configurations with the `kompose convert <file>` command, run the docker-compose files with the `kompose up` command, and more. If you want to know more about Kompose, we highly recommend that you see the documentation at this link: **https://kompose.io/**.

For this example, you'll these files in the AWS/EKS folder in the following GitHub repository: **https://github.com/ihuaylupo/manning-smia/tree/master/chapter12/AWS/EKS**. But before moving on, let's review the Kubernetes service types, which allow us to expose the services to external IP addresses. Kubernetes has the following four types of services:

- *ClusterIP*—Exposes the service on a cluster-internal IP. If we choose this option, our service is only going to be visible within our cluster.
- *NodePort*—Exposes the service at a static port (the `NodePort` value). Kubernetes allocates a default port range from 3000 to 32767. You can change this range by using the `--service-node-port-range` flag in the `spec.containers.commands` section of the service.yaml file.
- *LoadBalancer*—Exposes the service externally using a cloud load balancer.
- *ExternalName*—Maps the service to the contents of an external name.

If you take a look at the files, you'll see that some of the <*service*>.yml files have a `type=NodePort` and a `nodePort` attribute. In Kubernetes, if we don't define a service type, Kubernetes uses the default ClusterIP service type.

Now that we know more about the Kubernetes service types, let's continue. To create the services using these converted files, execute the following commands on the root of the AWS/ELK folder:

```
kubectl apply -f <service>.yaml,<deployment>.yaml
```

We prefer to execute this command individually to see that everything is created successfully, but you can also create all the services simultaneously. To do this, you need to concatenate all the YAML files with the `-f` argument as in the previous command. For example, to create the configuration server and view the pod status and the logs, we need to execute the following commands:

```
kubectl apply -f configserver-service.yaml,configserver-deployment.yaml
kubectl get pods
kubect logs <POD_NAME> --follow
```

To test that our service is up and running, we need to add a few rules to our security group to allow all the incoming traffic from the node ports. To do so, let's execute the following command to retrieve the security group ID:

```
aws ec2 describe-security-groups --filters Name=group-name,
Values="*eksctl-ostock-dev-cluster-nodegroup*"  --query "SecurityGroups[*].{Name:GroupName,ID:GroupId}"
```

With the security group ID, let's execute the following command to create an inbound rule. You can also do this in the AWS Management Console by going to the security group and creating a new inbound traffic rule like this:

```
aws ec2 authorize-security-group-ingress --protocol tcp --port 31000
--group-id [security-group-id] --cidr 0.0.0.0/0
```

To get the external IP, you can execute the `kubectl get nodes -o wide` command. This command shows you output similar to the one shown in figure 12.16.



Figure 12.16 Getting the external IP address for the node

Now you can open the following URL in your browser: http:<node-external-ip>: <NodePort>/actuator. In case you need to delete a pod, service, or deployment, you can execute one of the commands listed here:

```
kubectl delete -f <service>.yaml
kubectl delete -f <deployment>.yaml
kubectl delete <POD_NAME>
```

We are almost done. If you looked at the YAML files, you may have noted that the postgres.yaml file is a bit different. In this file, we'll specify that the database service will be used with an external address. In order to make the connection work, you should specify the endpoint for the RDS Postgres service. The following listing shows you how.

Listing 12.2 Adding the external reference to the database service

```
apiVersion: v1
kind: Service
metadata:
  labels:
    app: postgres-service
  name: postgres-service
spec:
  externalName: ostock-aws.cjuqpwnyahhy.us-east-2
                .rds.amazonaws.com                    (1)
  selector:
    app: postgres-service
  type: ExternalName
status:
  loadBalancer: {}
```

(1) Sets the RDS PostgreSQL endpoint

Make sure that you add the `externalName` endpoint of your RDS PostgreSQL instance. When you've made this change, you can execute the following command. Once the services are created and running, you can now continue running all of the other services.

```
kubectl apply -f postgres.yaml
```

While developing this chapter, we noticed that we needed to create a VPC peering connection and update the route tables of both the RDS and the EKS cluster to allow communication between them. We will not describe how to do this, but we highly recommend that you read the article, "Accessing Amazon RDS From AWS EKS," at **https://dev.to/bensooraj/accessing-amazon-rds-from-aws-eks-2pc3**, which contains all of the steps, and/or the official AWS documentation to understand a bit more about VPC peering at **https://docs.aws.amazon.com/vpc/latest/peering/create-vpc-peering-connection.html**.

At this point, you've successfully deployed your first set of services to an Amazon EKS cluster. Now, let's build on this by looking at how to design a build/deployment pipeline that automates the process of compiling, packaging, and deploying your services to Amazon.

12.4 Your build/deployment pipeline in action

From the general architecture laid out in section 12.1, you can see that there are many moving pieces behind a build/deployment pipeline. Because the purpose of this book is to show you things "in action," we're going to walk through the specifics of implementing a build/deployment pipeline for the O-stock services. Figure 12.17 lays out the different technologies we'll use to implement our pipeline.

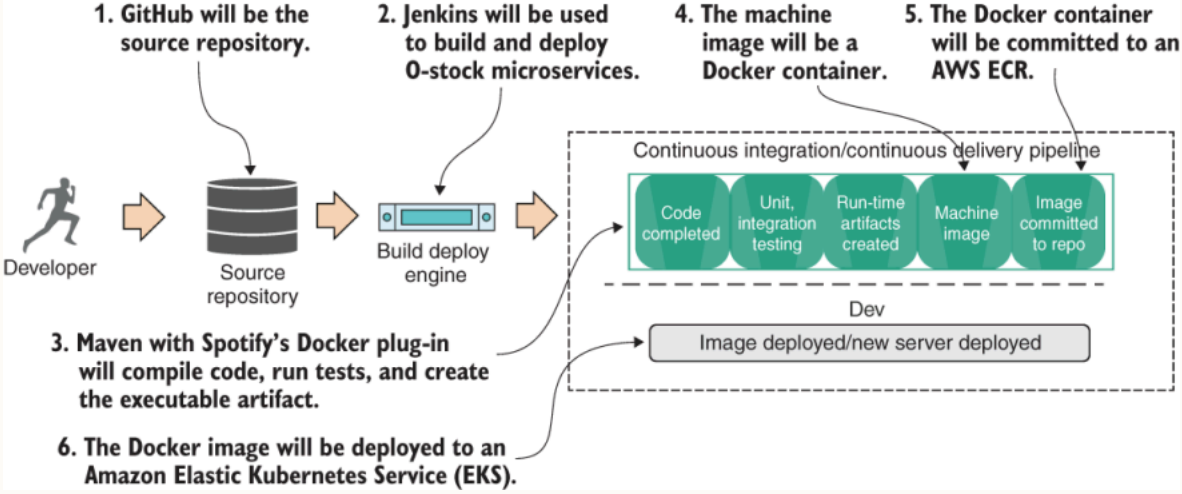


Figure 12.17 Technologies used in the O-stock build/deployment pipeline

Let's take a peek at the technologies we'll need for our build/deployment pipeline:

- *GitHub* (**http://github.com**)—GitHub is our source control repository for our source code. All the application code for this book is in GitHub. We chose GitHub as the source control repository because it offers a wide variety of webhooks and robust REST-based APIs for integrating GitHub into your build process.
- *Jenkins* (**https://www.jenkins.io/**)—Jenkins is the continuous integration engine we used for building and deploying the O-stock microservices and for provisioning the Docker image that will be deployed. Jenkins can be easily configured via its web interface and includes several built-in plugins that are going to make our work easier.
- *Maven/Spotify Docker plugin* (**https://github.com/spotify/dockerfile-maven**) *or Spring Boot with Docker* (**https://spring.io/guides/gs/spring-boot-docker/**)—While we use vanilla Maven to compile, test, and package Java code, these essential Maven plugins will allow us to kick off the creation of a Docker build right from within Maven.
- *Docker* (**https://www.docker.com/**)—We chose Docker as our container platform for two reasons. First, Docker is portable across multiple cloud providers. We can take the same Docker container and deploy it to AWS, Azure, or Cloud Foundry with a minimal amount of work. Second, Docker is lightweight.
  By the end of this book, you've built and deployed approximately 10 Docker containers (including a database server, messaging platform, and a search engine). Deploying the same number of virtual machines on a local desktop would be difficult because of the sheer size and speed of each image.
- *Amazon Elastic Container Registry (ECR)* (**https://aws.amazon.com/ecr/**)—After you build a service and create a Docker image, it's tagged with a unique identifier and pushed to a central repository. For this Docker image repository, we chose to use ECR.
- *Amazon's EKS Container Service* (**https://aws.amazon.com/eks/**)—The final destination for our microservices will be Docker instances deployed to Amazon's Docker platform. We chose Amazon as the cloud platform because it's by far the most mature of the cloud providers and makes it trivial to deploy Docker services.

12.5 Creating our build/deploy pipeline

Dozens of source control engines and build deploy engines (both on-premises and cloud-based) can implement your build/deployment pipeline. For the examples in this book, we purposely chose GitHub as the source control repository and Jenkins as the build engine. The Git source control repository is extremely popular, and GitHub is one of the largest cloud-based source control repositories available today. Jenkins is a build engine that integrates tightly with GitHub. It's also straightforward to use. Its simplicity and opinionated nature make it easy to get a simple build pipeline up and off the ground.

Up to now, all of the code examples in this book can be run solely from your desktop (with the exception of connectivity out to GitHub). For this chapter, if you want to completely follow the code examples, you'll need to set up your own GitHub, Jenkins, and AWS accounts. We're not going to walk through how to set up Jenkins; however, if you are not familiar with Jenkins, you can take a look at the file that we made that walks you through (step by step) how to set up the Jenkins environment from scratch in an EC2 instance. The file is located in the chapter 12 repository at the following link: **https://github.com/ihuaylupo/manning-smia/blob/master/chapter12/AWS/jenkins_Setup.md**.

12.5.1 Setting up GitHub

To construct our pipeline, we must create a GitHub webhook. What is a webhook? A webhook is also known as a web HTTP callback. These HTTP callbacks provide real-time information to other applications. Usually, the callback is triggered when a process or action is executed on the application containing the webhook.

If you haven't heard about webhooks, you might be wondering why we need them. For our purposes, we'll create a webhook in GitHub so that it lets Jenkins know when a code push is made. That way Jenkins can start a specific build process. To create a webhook in GitHub, we need to do the following steps. Figure 12.18 shows this process.

1. Go to the repository containing your pipeline projects
2. Click the Settings option
3. Select Webhooks
4. Click the Add Webhook button
5. Provide a specific payload URL
6. Click the Just the Push Event option
7. Click the Add Webhook button

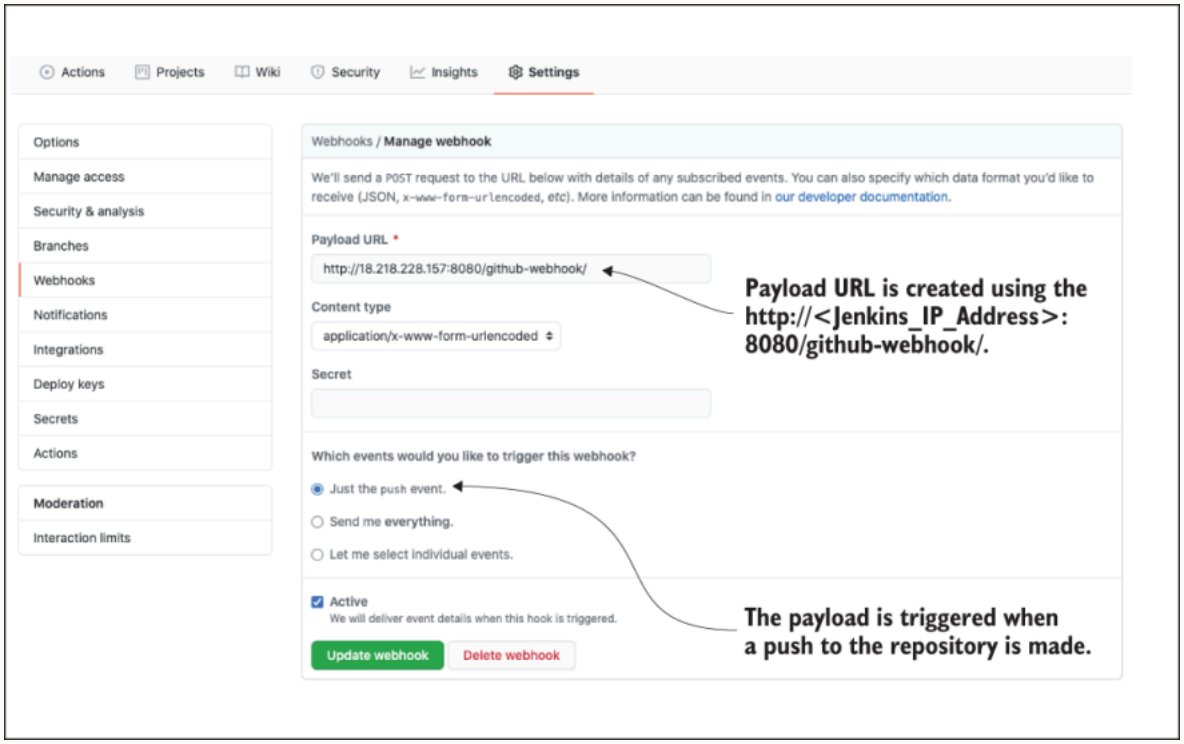


Figure 12.18 Generating a webhook for Jenkins

It's important to note that the payload URL is created using the IP address of Jenkins. This way GitHub can call Jenkins whenever a push to the repository is made. Now that we have the webhook, let's configure the pipeline using Jenkins.

12.5.2 Enabling our services to build in Jenkins

The heart of every service built in this book has been a Maven pom.xml file that's used to build the Spring Boot service, package it into an executable JAR, and build a Docker image that can be used to launch the service. Up until this chapter, the compilation and startup of the services occurred by

1. Opening a command-line window on your local machine.
2. Running the Maven script for the chapter. This builds all the services for the chapter and then packages them into a Docker image that's pushed to a locally running Docker repository.
3. Launching the newly created Docker images from your local Docker repository, by using the `docker-compose` and `docker-machine` commands to launch all the services for the chapter.

The question is, how do we repeat this process in Jenkins? It all begins with a single file called the Jenkinsfile. The Jenkinsfile is a script that describes the actions we want to run when Jenkins executes our build. This file is stored in the root directory of our microservices GitHub repository.

When a push occurs on a GitHub repository, the webhook calls Jenkins using the payload URL, and the Jenkins job searches for the Jenkinsfile to initiate the build process. Figure 12.19 shows the steps for this process.

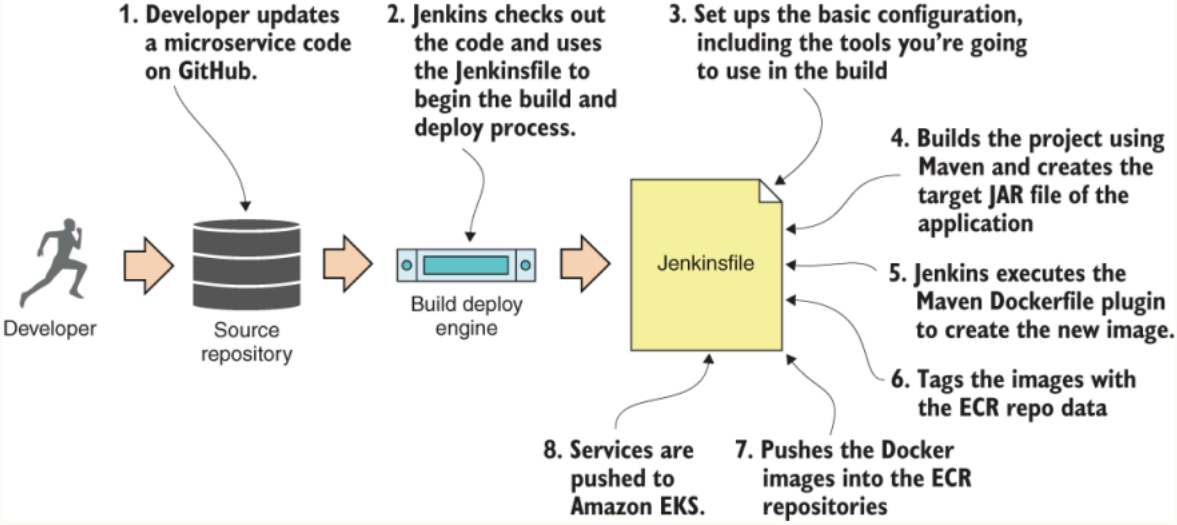


Figure 12.19 The steps taken—from developer to the Jenkinsfile—that build and deploy our software

As figure 12.19 shows:

1. A developer makes a change to one of the microservices in the GitHub repository.
2. Jenkins is notified by GitHub that a push has occurred. This notification is made by the GitHub webhook. Jenkins starts a process that is used to execute the build. Jenkins then checks out the source code from GitHub and uses the Jenkinsfile to begin the overall build and deployment process.
3. Jenkins sets up the basic configuration for the build and installs any dependencies.
4. Jenkins builds the project, executing the unit and integration tests, and generates the target JAR file for the application.
5. Jenkins executes the Maven Dockerfile plugin to create the new Docker images.
6. Jenkins then tags the new images with the ECR repository data.
7. The build process pushes the images to the ECR with the same tag name you used in step 6.
8. The build process connects to the EKS cluster and deploys the services using the service.yaml and deployment.yaml files.

A quick note

For this book, we set up separate folders in the same GitHub repository for each chapter. All the source code for the chapter can be built and deployed as a single unit. However, for your projects, we highly recommend that you set up each microservice in your environment with its own repository and independent build processes. That way, each service can be deployed independently of one another.

With the build process example, we deploy the configuration server as a single unit only because we want to push the projects to the Amazon cloud separately and to manage build scripts for each individual service. In this example, we only deploy the configuration server, but later, you can use the same steps to create other deployments.

Now that we've walked through the general steps involved in our build/deployment process with Jenkins, let's look at how to create the Jenkins pipeline.

NOTE To make the pipeline work, you need to install several Jenkins plugins such as GitHub Integration, GitHub, Maven Invoker, Maven Integration, Docker Pipeline, ECR, Kubernetes Continuous Deploy, and Kubernetes CLI. To learn more about these plugins, we recommend that you read the official Jenkins documentation for each (for example, for the Kubernetes CLI, see https://plugins.jenkins.io/kubernetes-cli/).

Once we've installed all the plugins, we need to create the Kubernetes and ECR credentials. First, let's start with the Kubernetes credentials. To create these, we need to go to Manage Jenkins, Manage Credentials, Jenkins, Global Credentials, and then click the Add Credentials file. Select the Kubernetes Configuration (kubeconfig) option and fill in the forms with your Kubernetes cluster information.

To retrieve the information from the Kubernetes cluster, connect to the Kubernetes cluster as we previously explained when we manually deployed our services and then execute the following command:

```
kubectl config view
```

Copy the returned contents and paste that into the Content text box in Jenkins. Figure 12.20 shows this step.

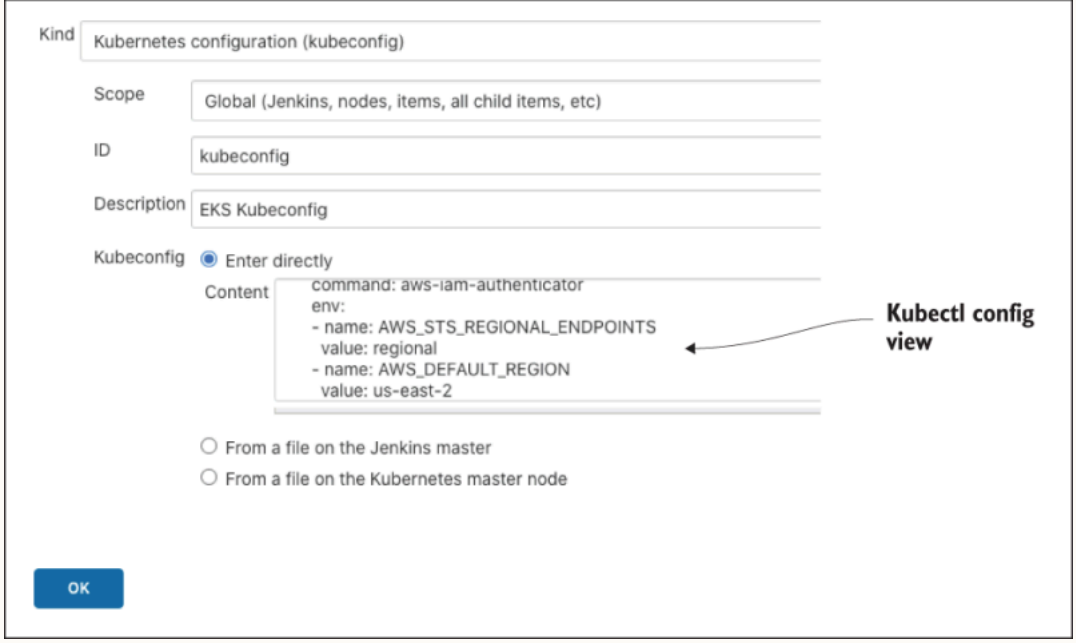


Figure 12.20 Configuring kubeconfig credentials to connect to the EKS cluster

Now, let's create the ECR credentials. In order to do this, we first need to go to the IAM page in the AWS console, then go to Users and click the Add User option. In the Add User page, you need to specify a username, the access type, and the policies. Next, download the .CSV file with the user credentials and, finally, save the user. We will guide you step by step as we create the credentials. First, add the following data in the IAM page:

User name: ecr-user
Access type: Programmatic access

In the second page, the Permissions page, select the Attach Existing Policies Directly option, and search for "AmazonEC2ContainerRegistryFullAccess" and then select it. Finally, on the last page, click Download .csv. The download step is really important because we'll need those credentials later. Finally, click the Save button.

The next step is to add the credentials to Jenkins, but before that, make sure you installed the ECR plugin in Jenkins. Once installed, go to Manage Jenkins, Manage Credentials, Jenkins, Global Configuration page and click the Add Credentials option. In this page, add the following data:

ID: ecr-user
Description: ECR User
Access Key ID: <Access_key_id_from_csv>
Secret Access Key: <Secret_access_key_from_csv>

Once the credentials are created, you can continue with the pipeline creation. Figure 12.21 shows the pipeline setup process. To access the page shown in figure 12.21, go to the Jenkins dashboard and click the New Item option in the top left of the screen.

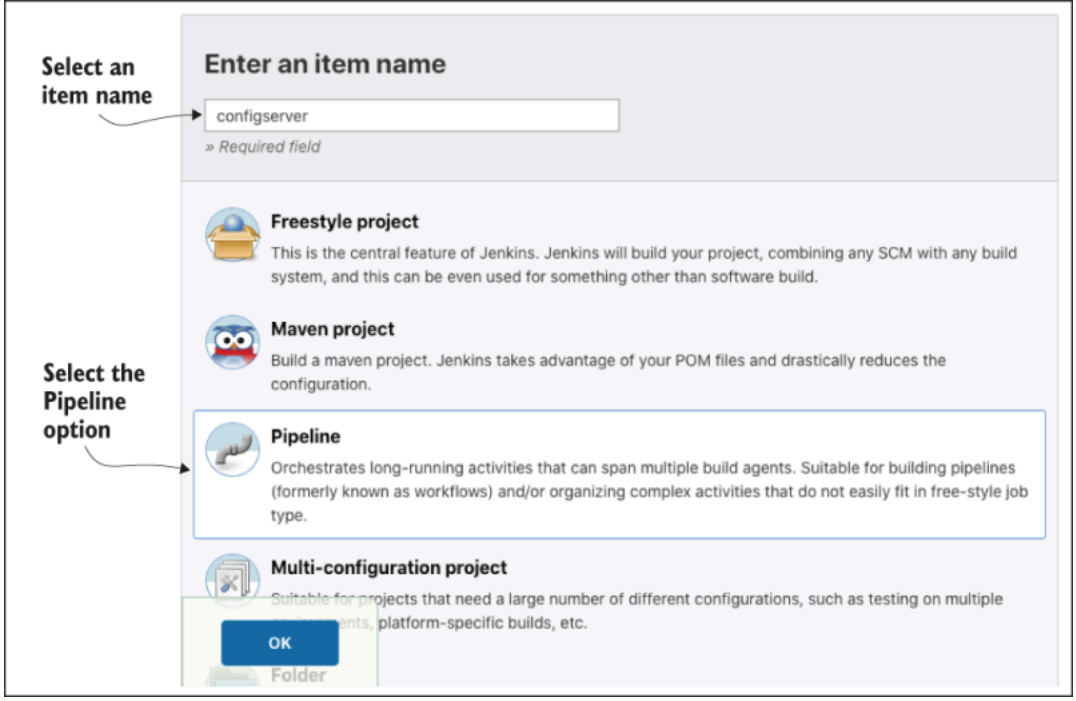


Figure 12.21 Creating a Jenkins pipeline to build the configuration server

The next step is to add the GitHub repository URL and select the GitHub Hook Trigger for GITScm Polling option (figure 12.22). This option enables the webhook payload we configured in the previous section.

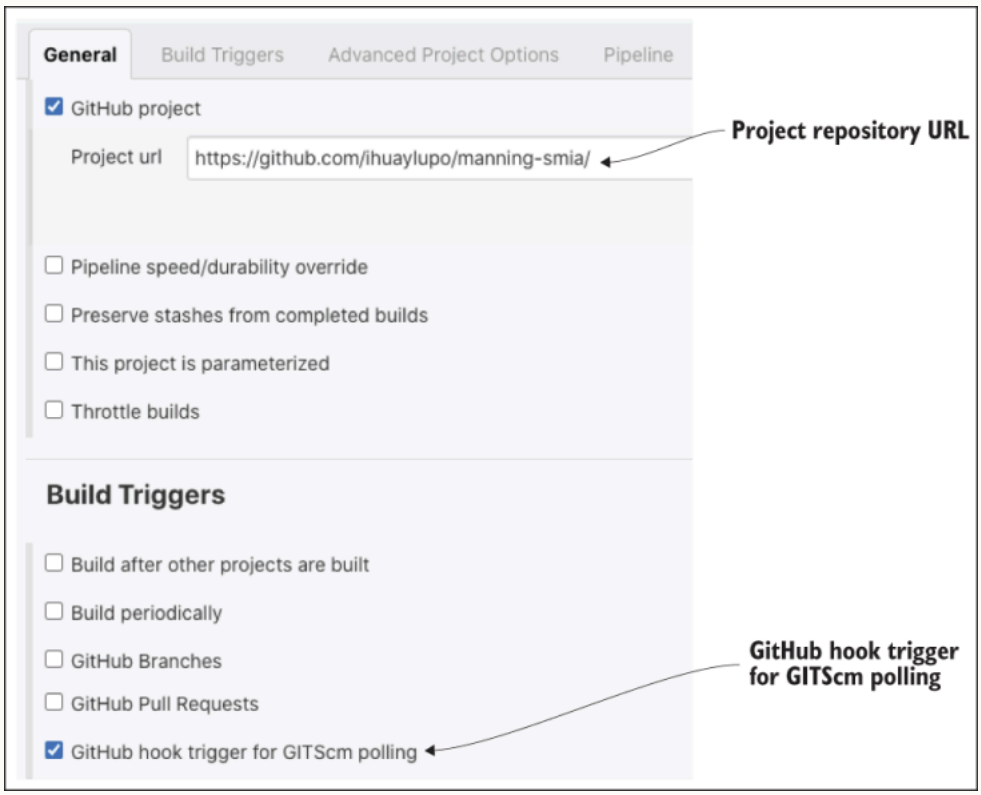


Figure 12.22 Configuring the GitHub webhook and repository URL in the Jenkins pipeline, step 1

Finally, the last step is to select Pipeline Script from SCM, which is located under the Pipeline section in the Definition drop-down list. This last step allows us to search for the Jenkinsfile in the same source code repository as the application. Figure 12.23 shows this process.

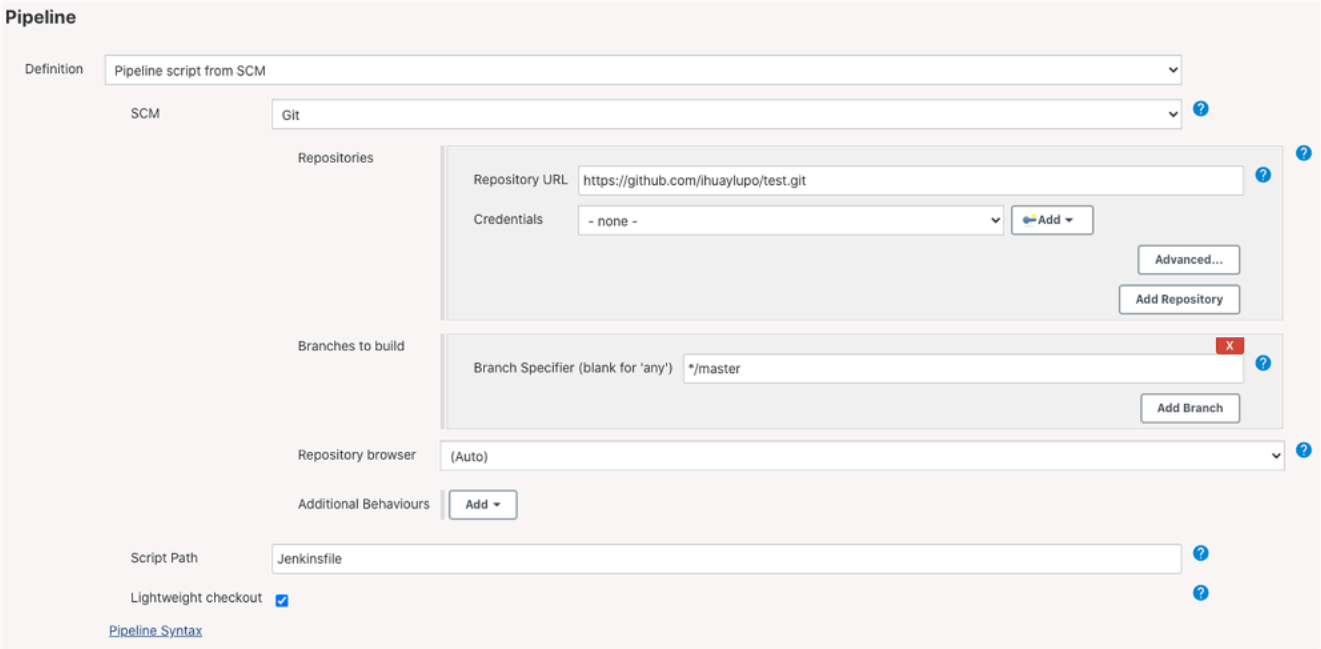


Figure 12.23 Configuring the GitHub webhook and repository URL in the Jenkins pipeline, step 2

In figure 12.23, we select Git from the SCM drop-down list and then add our pipeline-specific information such as the Repository URL, the branch and the script path that is the root (in our case), and the filename Jenkinsfile. Finally, click Save to go to the pipeline home page.

NOTE If your repository needs credentials, you'll see an error.

12.5.3 Understanding and generating the pipeline script

The Jenkinsfile deals with configuring the core run-time configuration of your build. Typically, this file is divided into several sections. Note that you can have as many sections or stages as you want. The next listing shows the Jenkinsfile for our configuration server.

Listing 12.3 Configuring the Jenkinsfile

```
node {
  def mvnHome
  stage('Preparation') {                                    1

    git 'https://github.com/ihuaylupo/spmia.git'            2
    mvnHome = tool 'M2_HOME'                                3
  }
  stage('Build') {                                          4
    // Run the maven build
    withEnv(["MVN_HOME=$mvnHome"]) {
      if (isUnix()) {
        sh "'${mvnHome}/bin/mvn' -Dmaven.test.failure.ignore
        clean package"                                      5
      } else {
        bat(/"%MVN_HOME%\bin\mvn" -Dmaven.test.failure.ignore
       clean package/)
      }
    }
  }
  stage('Results') {                                        6
    junit '**/target/surefire-reports/TEST-*.xml'
    archiveArtifacts 'configserver/target/*.jar'
  }
   stage('Build image') {                                   7
      sh "'${mvnHome}/bin/mvn'
-Ddocker.image.prefix=8193XXXXXXX43.dkr.ecr.us-east-2.amazonaws.com/
 ostock -Dproject.artifactId=configserver
-Ddocker.image.version=latest dockerfile:build"             8
   }
   stage('Push image') {                                    9
      docker.withRegistry('https://8193XXXXXX43.dkr.ecr.us-east-
```

```
        2.amazonaws.com', 'ecr:us-east-2:ecr-user') {
          sh "docker push 8193XXXXXX43.dkr.ecr.us-east-
          2.amazonaws.com/ostock/
          configserver:latest"                          ⑩
        }
      }
    stage('Kubernetes deploy') {                        ⑪
      kubernetesDeploy configs: 'configserver-deployment.yaml', kubeConfig: [path: ''], kubeconfigId: 'kubeconfig', secretName: '', ssh: [sshCredentialsId:
    }

}
```

❶ Defines the preparation stage

❷ Sets the GIT repository

❸ Exposes the Maven configuration from the Jenkins server

❹ Defines the build stage

❺ Executes the Maven clean package goals

❻ Obtains the JUnit results and archives the application JAR file that we need for the Docker image

❼ Defines the build image stage

❽ Executes the Maven dockerfile:build goal, which creates the new Docker image and tags it with the ECR repository information

❾ Defines the push image stage

❿ Pushes the images into the ECR repository

⓫ Defines the Kubernetes deploy stage

This Jenkinsfile describes all of the steps involved in our pipeline. As you can see, the file contains the commands to execute each stage. In the first stage, we add the Git configuration. Doing this, we can remove the step where we define the GitHub URL repository in the build section. In the second stage, the second command line exposes the Maven installation path so that we can use the `maven` command throughout our Jenkinsfile. In the third stage, we define the command we will use to package our application. In this case, we use the clean package goals. For packing our application, we'll use the Maven environment variable where we define the Maven root directory.

The fourth stage allows us to execute the `dockerfile:build` Maven goal that is in charge of creating the new Docker image. If you remember, we defined the Maven spotify-dockerfile plugin in the pom.xml files of our microservices. If you take a closer look at the line with `dockerfile:build`, you'll notice that we send some parameters to Maven. Let's take a look:

```
sh "'${mvnHome}/bin/mvn' -Ddocker.image.prefix=8193XXXXXXX43.dkr.ecr.us-
east-2.amazonaws.com/ostock -Dproject.artifactId=configserver
-Ddocker.image.version=latest dockerfile:build"
```

In this command, we pass the Docker image prefix, the image version, and the artifact ID. But here you can send as many parameters as you want. Also, here is a good place to set the profile you are going to use in the deployment (for example: dev, stage, or production). Also, in this stage, you can create and assign specific tags with the ECR repository data to the new images. In this particular scenario, we use the latest version, but here you can specify different variables to create different Docker images.

The fifth stage pushes the images to the ECR repositories. In this case, we added the configuration server just created to the Docker image in the 8193XXXXXXX43.dkr .ecr.us-east-2.amazonaws.com/ostock repository. The final stage, the Kubernetes deploy stage, deploys to our EKS cluster. This command can vary, depending on your configuration, so we will give you a tip on how to create the pipeline script for this stage automatically.

12.5.4 Creating the Kubernetes pipeline scripts

Jenkins offers a Pipeline Syntax option that lets us automatically generate different pipeline snippets that we can use in our Jenkinsfile. Figure 12.24 shows this process.

To use this option, we need to execute the following steps:

1. Go to the Jenkins dashboard page and select your pipeline.
2. Click the Pipeline Syntax option shown in the left column on the dashboard page.
3. On the Snippet Generator page, click the Sample Step drop-down list and select the kubernetesDeploy: Deploy to Kubernetes option.
4. In the Kubeconfig drop-down list, select the Kubernetes credentials that we created in the previous section.
5. In the Config Files option, select the deployment.yaml file. This file is located in the root of the GitHub project.
6. Leave the other values as default.
7. Click the Generate Pipeline Script button.

NOTE Remember, for this book, we set up separate folders in the same GitHub repository for each chapter of the book. All the source code for this chapter can be built and deployed as a single unit. However, we highly recommend that you set up each microservice in your environment with its own repository and with its own independent build processes.

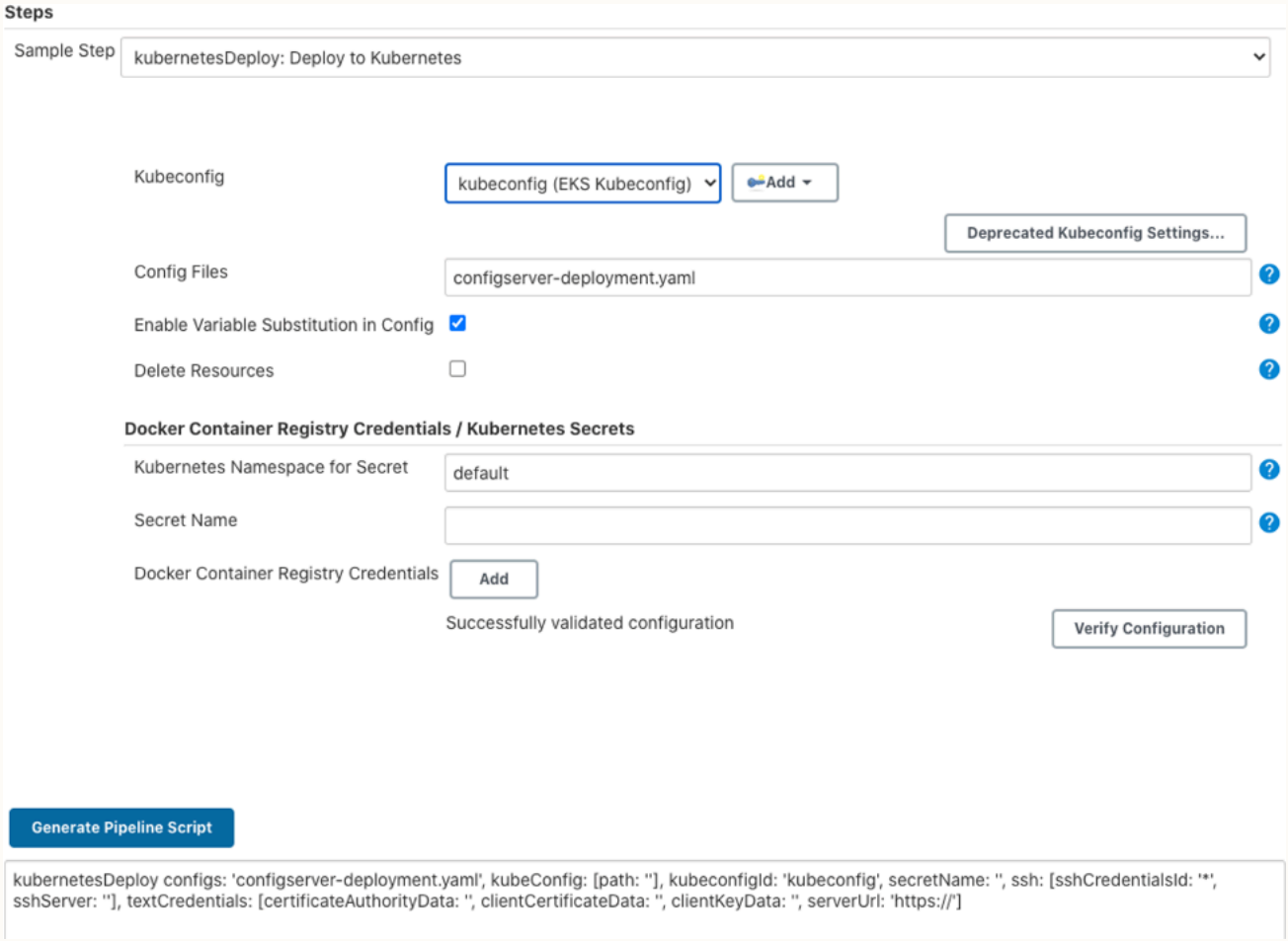


Figure 12.24 Using the Pipeline Syntax option to generate the `kubernetesDeploy` command

Once, everything is set up, you can now change your code. Your Jenkins pipeline is automatically triggered.

12.6 Closing thoughts on the build/deployment pipeline

As this chapter closes (and the book), we hope you've gained an appreciation for the amount of work that goes into building a build/deployment pipeline. A well-functioning build/deployment pipeline is critical to the deployment of services. And the success of your microservices architecture depends on more than just the code:

- Understand that the code in this build/deploy pipeline is simplified for the purposes of demonstration in this book. A good build/deployment pipeline is much more generalized. It will be supported by the DevOps team and broken into a series of independent steps (compile > package > deploy > test) that the development teams can use to "hook" their microservice build scripts into.

- The virtual machine imaging process used in this chapter is simplistic. Each microservice is built using a Dockerfile to define the software that's going to be installed on the Docker container. Many shops use provisioning tools like Ansible (https://github.com/ansible/ansible), Puppet (https://github.com/puppetlabs/puppet), or Chef (https://github.com/chef/chef) to install and configure the operating systems on the virtual machine or the container images.
- The cloud deployment topology for your application has been consolidated to a single server. In the real build/deployment pipeline, each microservice would have its own build scripts and would be deployed independently of any others to a cluster EKS container.

Writing microservices is a challenging task. With microservices, the complexity of creating applications doesn't go away; it is just transformed. The concepts around building individual microservices are easy to understand, but running and supporting a robust microservice architecture involves more than writing the code. Remember, a microservices architecture gives us a lot of options and decisions to make on how to create our application. When making these decisions, we need to consider that microservices are loosely coupled, abstract, independent, and constrained.

Hopefully, by now, we've given you enough information (and experiences) to create your own microservices architecture. In this book, you learned what a microservice is and the critical design decisions needed to operationalize them. You also learned how to create a complete microservices architecture following the twelve-factor best practices (configuration management, service discovery, messaging, logging, tracing, security), using a variety of tools and technologies that can be combined perfectly with Spring to deliver a robust microservices environment. And, finally, you learned how to implement a build/deployment pipeline.

All the previously mentioned ingredients create the perfect recipe to develop successful microservices. In closing, as Martin Fowler says, "If we don't create good architecture, then, in the end, we're deceiving our customers because we're slowing down their ability to compete."

ummary

- The build/deployment pipeline is a critical part of delivering microservices. A well-functioning build/deployment pipeline should allow new features and bug fixes to be deployed in minutes.
- The build/deployment pipeline should be automated with no direct human interaction to deliver a service. Any manual part of the process represents an opportunity for variability and failure.
- The build/deployment pipeline automation requires a great deal of scripting and configuration to get right. The amount of work needed to build it shouldn't be underestimated.
- The build/deployment pipeline should deliver an immutable virtual machine or container image. Once a server image is created, it should never be modified.
- Environment-specific server configuration should be passed in as parameters at the time the server is set up.

Previous chapter
‹ 11 Distributed tracing with Spring Cloud Sleuth and Zipkin

Next chapter
appendix A. Microservices architecture best practices ›